МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Кафедра автоматизованих систем управління

ЗВІТ

до лабораторної роботи №10

з дисципліни «Програмування мобільних додатків»

на тему: «Локальне збереження даних у Flutter-застосунку»

Виконав: студент групи КН-2327Б
Багрій-Белз А. В.

Прийняв: доцент кафедри АСУ

Львів — 2026

Лабораторна робота №10

Тема: Локальне збереження даних у Flutter-застосунку

Мета роботи: ознайомитися зі способами локального збереження даних у Flutter, навчитися зберігати та завантажувати дані користувача з екрана реєстрації, а також зберігати список обраних елементів (favorites ids) за допомогою пакета SharedPreferences для подальшого відображення та збереження стану між перезапусками застосунку.

Завдання на лабораторну роботу:

1. Додати у проєкт пакет для локального збереження даних (SharedPreferences).
2. Реалізувати збереження даних користувача з екрана реєстрації.
3. Зберігати такі поля: a) ім’я користувача; b) email; c) пароль або інше поле, передбачене формою реєстрації.
4. Реалізувати завантаження даних користувача після повторного запуску застосунку.
5. Відобразити збережені дані на екрані ProfileScreen.
6. Реалізувати додавання елементів у список обраного.
7. Зберігати локально лише список favorites ids.
8. Після запуску застосунку автоматично відновлювати список обраних елементів за збереженими id.
9. На екрані FavoritesScreen відображати лише ті елементи, id яких містяться у списку збережених.
10. Перевірити, що після перезапуску застосунку дані профілю не втрачаються, а обрані елементи залишаються в списку Favorites.

1. Написання коду-розв’язку завдання

Для виконання лабораторної роботи розроблено кросплатформений мобільний застосунок **storage_app** з модульною трирівневою архітектурою:

• **Шар даних та сервісів (Data & Services Layer):** сервіс StorageService на основі SharedPreferences для асинхронного збереження/завантаження ключів ім'я, email, пароля, телефону, статусу та списку рядкових ID favorite_ids.

• **Шар моделей (Models Layer):** моделі UserModel (ім'я, email, пароль, контакти, дата реєстрації) та CourseItem (каталог освітніх IT-курсів з ідентифікаторами, метаданими, рейтингом та категоріями).

• **Шар інтерфейсу (Presentation Layer):** RegistrationScreen (валідація та збереження форми реєстрації), CatalogScreen (каталог курсів з кнопками додавання в обране), FavoritesScreen (фільтрація виключно за збереженими ID), ProfileScreen (відображення збережених даних користувача, пароля та статистики) та MainNavigationScreen (навігація між вкладками через NavigationBar).

1.1. Конфігураційний файл проєкту (pubspec.yaml):

```
name: storage_app
description: "Лабораторна робота №10: Локальне збереження даних у Flutter-застосунку"
publish_to: 'none'
version: 1.0.0+1


environment:
  sdk: ^3.13.2


dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.9
  shared_preferences: ^2.5.6


dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0


flutter:
  uses-material-design: true
```

1.2. Модель даних користувача (lib/models/user_model.dart):

```
/// Модель користувача для збереження та завантаження через SharedPreferences
class UserModel {
  final String name;
  final String email;
  final String password;
  final String phone;
  final String bio;
  final String registeredAt;


  const UserModel({
    required this.name,
    required this.email,
    required this.password,
    this.phone = '+380 67 123 45 67',
    this.bio = 'Студент НУ «Львівська політехніка», група КН-2327Б',
    this.registeredAt = '',
  });


  bool get isEmpty => name.isEmpty && email.isEmpty;
  bool get isNotEmpty => !isEmpty;


  /// Створення копії з можливістю оновлення окремих полів
  UserModel copyWith({
    String? name,
    String? email,
    String? password,
    String? phone,
    String? bio,
    String? registeredAt,
  }) {
    return UserModel(
      name: name ?? this.name,
      email: email ?? this.email,
      password: password ?? this.password,
      phone: phone ?? this.phone,
      bio: bio ?? this.bio,
      registeredAt: registeredAt ?? this.registeredAt,
    );
  }


  /// Порожній користувач (стан до реєстрації)
  static const UserModel empty = UserModel(
    name: '',
    email: '',
    password: '',
    phone: '',
    bio: '',
    registeredAt: '',
  );
}
```

1.3. Модель навчальних курсів каталогу (lib/models/course_item.dart):

```
import 'package:flutter/material.dart';


/// Модель елемента каталогу (навчальні IT-курси)
class CourseItem {
```

```
  final String id;
  final String title;
  final String category;
  final String description;
  final double rating;
  final int studentsCount;
  final String duration;
  final IconData icon;
  final Color accentColor;


  const CourseItem({
    required this.id,
    required this.title,
    required this.category,
    required this.description,
    required this.rating,
    required this.studentsCount,
    required this.duration,
    required this.icon,
    required this.accentColor,
  });


  /// Демонстраційний перелік курсів у каталозі
  static const List<CourseItem> sampleCourses = [
    CourseItem(
      id: 'course_1',
      title: 'Розробка мобільних додатків на Flutter & Dart',
      category: 'Mobile Dev',
      description:
          'Поглиблений курс з побудови кросплатформених мобільних застосунків, state management
та архітектури.',
      rating: 4.9,
      studentsCount: 1240,
      duration: '48 год',
      icon: Icons.flutter_dash,
      accentColor: Color(0xFF02569B),
    ),
    CourseItem(
      id: 'course_2',
      title: 'Архітектура програмного забезпечення: BLoC & Clean Arch',
      category: 'Architecture',
      description:
          'Проєктування масштабованих корпоративних додатків, шар даних, репозиторії та інверсія
залежностей.',
      rating: 4.8,
      studentsCount: 890,
      duration: '36 год',
      icon: Icons.account_tree_outlined,
      accentColor: Color(0xFF6C5CE7),
    ),
    CourseItem(
      id: 'course_3',
      title: 'UI/UX дизайн та дизайн-системи у Figma & Flutter',
      category: 'Design',
      description:
          'Створення інтерактивних інтерфейсів, токенти дизайну, Material 3 та анімації
переходів.',
      rating: 4.95,
      studentsCount: 1560,
      duration: '28 год',
      icon: Icons.palette_outlined,
      accentColor: Color(0xFFFD79A8),
```

```
    ),
    CourseItem(
      id: 'course_4',
      title: 'Локальні бази даних та кешування: SQLite, Hive, SharedPreferences',
      category: 'Data Storage',
      description:
          'Робота з key-value сховищами, локальними реляційними базами даних та офлайн-режимом у
мобільних додатках.',
      rating: 4.85,
      studentsCount: 1020,
      duration: '32 год',
      icon: Icons.storage_rounded,
      accentColor: Color(0xFF00B894),
    ),
    CourseItem(
      id: 'course_5',
      title: 'Кібербезпека мобільних платформ та криптографія',
      category: 'Security',
      description:
          'Захищене збереження токенів, шифрування SharedPreferences
(EncryptedSharedPreferences) та захист від reverse-engineering.',
      rating: 4.75,
      studentsCount: 670,
      duration: '40 год',
      icon: Icons.security_rounded,
      accentColor: Color(0xFFE17055),
    ),
    CourseItem(
      id: 'course_6',
      title: 'DevOps, CI/CD та тестування у Flutter (Unit, Widget, Integration)',
      category: 'DevOps',
      description:
          'Автоматизація збірки APK/AAB, налаштування GitHub Actions, Fastlane та покриття коду
тестами.',
      rating: 4.9,
      studentsCount: 810,
      duration: '30 год',
      icon: Icons.integration_instructions_outlined,
      accentColor: Color(0xFF0984E3),
    ),
  ];
}
```

1.4. Сервіс локального збереження даних (lib/services/storage_service.dart):

```
import 'package:shared_preferences/shared_preferences.dart';
import '../models/user_model.dart';


/// Сервіс для роботи з локальним сховищем SharedPreferences (п. 1-8 ТЗ)
class StorageService {
  // Ключі для збереження даних користувача (п. 3 ТЗ)
  static const String keyName = 'user_name';
  static const String keyEmail = 'user_email';
  static const String keyPassword = 'user_password';
  static const String keyPhone = 'user_phone';
  static const String keyBio = 'user_bio';
  static const String keyRegisteredAt = 'user_registered_at';
  static const String keyIsRegistered = 'is_registered';
```

```
  // Ключ для збереження списку favorites ids (п. 7 ТЗ)
  static const String keyFavoriteIds = 'favorite_ids';


  /// Збереження даних користувача з екрана реєстрації (п. 2, 3 ТЗ)
  Future<void> saveUser(UserModel user) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(keyName, user.name);
    await prefs.setString(keyEmail, user.email);
    await prefs.setString(keyPassword, user.password);
    await prefs.setString(keyPhone, user.phone);
    await prefs.setString(keyBio, user.bio);
    await prefs.setString(
      keyRegisteredAt,
      user.registeredAt.isNotEmpty
          ? user.registeredAt
          : DateTime.now().toLocal().toString().split('.')[0],
    );
    await prefs.setBool(keyIsRegistered, true);
  }


  /// Завантаження даних користувача після повторного запуску (п. 4, 5 ТЗ)
  Future<UserModel?> loadUser() async {
    final prefs = await SharedPreferences.getInstance();
    final isRegistered = prefs.getBool(keyIsRegistered) ?? false;
    final name = prefs.getString(keyName) ?? '';
    final email = prefs.getString(keyEmail) ?? '';


    if (!isRegistered && name.isEmpty && email.isEmpty) {
      return null;
    }


    return UserModel(
      name: name,
      email: email,
      password: prefs.getString(keyPassword) ?? '',
      phone: prefs.getString(keyPhone) ?? '+380 67 123 45 67',
      bio: prefs.getString(keyBio) ??
          'Студент НУ «Львівська політехніка», група КН-2327Б',
      registeredAt: prefs.getString(keyRegisteredAt) ?? '',
    );
  }


  /// Перевірка чи зареєстрований користувач
  Future<bool> isUserRegistered() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getBool(keyIsRegistered) ?? false;
  }


  /// Видалення облікового запису (вихід)
  Future<void> clearUser() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.remove(keyName);
    await prefs.remove(keyEmail);
    await prefs.remove(keyPassword);
    await prefs.remove(keyPhone);
    await prefs.remove(keyBio);
    await prefs.remove(keyRegisteredAt);
    await prefs.setBool(keyIsRegistered, false);
  }
```

```
  /// Завантаження збереженого списку favorites ids (п. 7, 8 ТЗ)
  Future<List<String>> loadFavoriteIds() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getStringList(keyFavoriteIds) ?? [];
  }

  /// Збереження оновленого списку favorites ids (п. 6, 7 ТЗ)
  Future<void> saveFavoriteIds(List<String> favoriteIds) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setStringList(keyFavoriteIds, favoriteIds);
  }

  /// Додавання або видалення елемента зі списку обраного (Toggle)
  Future<bool> toggleFavorite(String id) async {
    final ids = await loadFavoriteIds();
    final mutableIds = List<String>.from(ids);
    final isFav = mutableIds.contains(id);

    if (isFav) {
      mutableIds.remove(id);
    } else {
      mutableIds.add(id);
    }

    await saveFavoriteIds(mutableIds);
    return !isFav; // true якщо додано, false якщо видалено
  }

  /// Перевірка чи конкретний id входить до обраного
  Future<bool> isFavorite(String id) async {
    final ids = await loadFavoriteIds();
    return ids.contains(id);
  }

  /// Очищення списку обраного
  Future<void> clearFavorites() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.remove(keyFavoriteIds);
  }

  /// Повне очищення всіх локальних даних
  Future<void> clearAll() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.clear();
  }
}
```

1.5. Палітра кольорів інтерфейсу (lib/constants/app_colors.dart):

```
import 'package:flutter/material.dart';

class AppColors {
  static const Color primary = Color(0xFF1E88E5);
```

```
  static const Color primaryDark = Color(0xFF1565C0);
  static const Color secondary = Color(0xFFFF9800);
  static const Color background = Color(0xFFF5F7FB);
  static const Color surface = Colors.white;
  static const Color textPrimary = Color(0xFF263238);
  static const Color textSecondary = Color(0xFF78909C);
  static const Color border = Color(0xFFCFD8DC);
  static const Color success = Color(0xFF43A047);
  static const Color error = Color(0xFFE53935);
  static const Color favorite = Color(0xFFE91E63);
}
```

1.6. Екран реєстрації користувача (lib/screens/registration_screen.dart):

```
import 'package:flutter/material.dart';
import '../constants/app_colors.dart';
import '../models/user_model.dart';
import '../services/storage_service.dart';


/// Екран реєстрації користувача (п. 2, 3 ТЗ)
/// Зберігає ім'я, email, пароль та супутні поля у SharedPreferences
class RegistrationScreen extends StatefulWidget {
  final StorageService storageService;
  final VoidCallback onRegistered;
  final UserModel? initialUser;


  const RegistrationScreen({
    super.key,
    required this.storageService,
    required this.onRegistered,
    this.initialUser,
  });


  @override
  State<RegistrationScreen> createState() => _RegistrationScreenState();
}


class _RegistrationScreenState extends State<RegistrationScreen> {
  final _formKey = GlobalKey<FormState>();


  late final TextEditingController _nameController;
  late final TextEditingController _emailController;
  late final TextEditingController _passwordController;
  late final TextEditingController _phoneController;
  late final TextEditingController _bioController;


  bool _obscurePassword = true;
  bool _isLoading = false;


  @override
  void initState() {
    super.initState();
    final init = widget.initialUser;
    _nameController = TextEditingController(text: init?.name ?? '');
    _emailController = TextEditingController(text: init?.email ?? '');
    _passwordController = TextEditingController(text: init?.password ?? '');
```

```
    _phoneController = TextEditingController(
      text: init?.phone ?? '+380 67 123 45 67',
    );
    _bioController = TextEditingController(
      text: init?.bio ?? 'Студент НУ «Львівська політехніка», група КН-2327Б',
    );
  }

  @override
  void dispose() {
    _nameController.dispose();
    _emailController.dispose();
    _passwordController.dispose();
    _phoneController.dispose();
    _bioController.dispose();
    super.dispose();
  }

  void _fillDemoData() {
    setState(() {
      _nameController.text = 'Багрій-Белз Андрій';
      _emailController.text = 'andrii.bagriy@nulp.ua';
      _passwordController.text = 'SecurePass2026!';
      _phoneController.text = '+380 67 987 65 43';
      _bioController.text = 'Студент групи КН-2327Б | Лабораторна робота №10';
    });
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Демо-дані успішно підставлено у форму'),
        duration: Duration(seconds: 1),
      ),
    );
  }

  Future<void> _submitRegistration() async {
    if (!_formKey.currentState!.validate()) {
      return;
    }

    setState(() => _isLoading = true);

    final user = UserModel(
      name: _nameController.text.trim(),
      email: _emailController.text.trim(),
      password: _passwordController.text.trim(),
      phone: _phoneController.text.trim(),
      bio: _bioController.text.trim(),
      registeredAt: DateTime.now().toLocal().toString().split('.')[0],
    );

    // Збереження даних користувача у SharedPreferences (п. 2, 3 ТЗ)
    await widget.storageService.saveUser(user);

    if (!mounted) return;
    setState(() => _isLoading = false);

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Дані успішно збережено в SharedPreferences!'),
```

```
        backgroundColor: AppColors.success,
        duration: Duration(seconds: 2),
      ),
    );

    widget.onRegistered();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppColors.background,
      appBar: AppBar(
        title: const Text('Реєстрація користувача'),
        centerTitle: true,
        backgroundColor: AppColors.surface,
        elevation: 0.5,
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(20.0),
          child: Form(
            key: _formKey,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                // Іконка та заголовок
                Center(
                  child: Container(
                    padding: const EdgeInsets.all(16),
                    decoration: BoxDecoration(
                      color: AppColors.primary.withValues(alpha: 0.1),
                      shape: BoxShape.circle,
                    ),
                    child: const Icon(
                      Icons.person_add_alt_1_rounded,
                      size: 50,
                      color: AppColors.primary,
                    ),
                  ),
                ),
                const SizedBox(height: 12),
                const Text(
                  'Створення профілю',
                  textAlign: TextAlign.center,
                  style: TextStyle(
                    fontSize: 22,
                    fontWeight: FontWeight.bold,
                    color: AppColors.textPrimary,
                  ),
                ),
                const SizedBox(height: 6),
                const Text(
                  'Введені дані будуть локально збережені у SharedPreferences та
відновлюватимуться при повторному запуску',
                  textAlign: TextAlign.center,
                  style: TextStyle(fontSize: 13, color: AppColors.textSecondary),
                ),
                const SizedBox(height: 24),

                // 1. Поле імені користувача (п. 3.а)
                TextFormField(
```

```
                key: const Key('name_field'),
                controller: _nameController,
                decoration: InputDecoration(
                  labelText: 'Ім’я користувача *',
                  hintText: 'Введіть ваше ім’я або ПІБ',
                  prefixIcon: const Icon(Icons.badge_outlined),
                  filled: true,
                  fillColor: AppColors.surface,
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(12),
                  ),
                ),
                validator: (value) {
                  if (value == null || value.trim().isEmpty) {
                    return 'Будь ласка, вкажіть ім’я користувача';
                  }
                  if (value.trim().length < 2) {
                    return 'Ім’я має містити щонайменше 2 символи';
                  }
                  return null;
                },
              ),
              const SizedBox(height: 16),

              // 2. Поле email (п. 3.b)
              TextFormField(
                key: const Key('email_field'),
                controller: _emailController,
                keyboardType: TextInputType.emailAddress,
                decoration: InputDecoration(
                  labelText: 'Електронна пошта (email) *',
                  hintText: 'example@domain.com',
                  prefixIcon: const Icon(Icons.email_outlined),
                  filled: true,
                  fillColor: AppColors.surface,
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(12),
                  ),
                ),
                validator: (value) {
                  if (value == null || value.trim().isEmpty) {
                    return 'Будь ласка, введіть email';
                  }
                  if (!value.contains('@') || !value.contains('.')) {
                    return 'Введіть коректну адресу email';
                  }
                  return null;
                },
              ),
              const SizedBox(height: 16),

              // 3. Поле пароля (п. 3.c)
              TextFormField(
                key: const Key('password_field'),
                controller: _passwordController,
                obscureText: _obscurePassword,
                decoration: InputDecoration(
                  labelText: 'Пароль *',
                  hintText: 'Введіть безпечний пароль',
                  prefixIcon: const Icon(Icons.lock_outline),
                  suffixIcon: IconButton(
                    icon: Icon(
```

```
            _obscurePassword
                ? Icons.visibility_off
                : Icons.visibility,
          ),
          onPressed: () {
            setState(() => _obscurePassword = !_obscurePassword);
          },
        ),
        filled: true,
        fillColor: AppColors.surface,
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12),
        ),
      ),
      validator: (value) {
        if (value == null || value.trim().isEmpty) {
          return 'Будь ласка, введіть пароль';
        }
        if (value.trim().length < 4) {
          return 'Пароль має містити не менше 4 символів';
        }
        return null;
      },
    ),
    const SizedBox(height: 16),

    // 4. Додаткове поле номера телефону
    TextFormField(
      key: const Key('phone_field'),
      controller: _phoneController,
      keyboardType: TextInputType.phone,
      decoration: InputDecoration(
        labelText: 'Номер телефону',
        prefixIcon: const Icon(Icons.phone_outlined),
        filled: true,
        fillColor: AppColors.surface,
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12),
        ),
      ),
    ),
    const SizedBox(height: 16),

    // 5. Поле опису/статусу/групи
    TextFormField(
      key: const Key('bio_field'),
      controller: _bioController,
      maxLines: 2,
      decoration: InputDecoration(
        labelText: 'Статус / Навчальна група',
        prefixIcon: const Icon(Icons.school_outlined),
        filled: true,
        fillColor: AppColors.surface,
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12),
        ),
      ),
    ),
    const SizedBox(height: 24),

    // Кнопка реєстрації
    ElevatedButton(
```

```
                  key: const Key('register_button'),
                  style: ElevatedButton.styleFrom(
                    backgroundColor: AppColors.primary,
                    foregroundColor: Colors.white,
                    padding: const EdgeInsets.symmetric(vertical: 15),
                    shape: RoundedRectangleBorder(
                      borderRadius: BorderRadius.circular(12),
                    ),
                    elevation: 2,
                  ),
                  onPressed: _isLoading ? null : _submitRegistration,
                  child: _isLoading
                      ? const SizedBox(
                          height: 20,
                          width: 20,
                          child: CircularProgressIndicator(
                            strokeWidth: 2,
                            color: Colors.white,
                          ),
                        )
                      : const Text(
                          'Зареєструватися та зберегти дані',
                          style: TextStyle(
                            fontSize: 16,
                            fontWeight: FontWeight.bold,
                          ),
                        ),
                ),
                const SizedBox(height: 12),


                // Кнопка швидкого демо-заповнення
                OutlinedButton.icon(
                  key: const Key('demo_fill_button'),
                  style: OutlinedButton.styleFrom(
                    foregroundColor: AppColors.primary,
                    side: const BorderSide(color: AppColors.primary),
                    padding: const EdgeInsets.symmetric(vertical: 13),
                    shape: RoundedRectangleBorder(
                      borderRadius: BorderRadius.circular(12),
                    ),
                  ),
                  icon: const Icon(Icons.auto_fix_high),
                  label: const Text('Заповнити демо-даними (КН-2327Б)'),
                  onPressed: _fillDemoData,
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

1.7. Екран каталогу курсів (lib/screens/catalog_screen.dart):

```
import 'package:flutter/material.dart';
import '../constants/app_colors.dart';
import '../models/course_item.dart';
```

```
/// Екран каталогу курсів (п. 6, 7 ТЗ)
/// Дозволяє додавати/видаляти курси з обраного з миттєвим збереженням id
class CatalogScreen extends StatelessWidget {
  final List<String> favoriteIds;
  final ValueChanged<String> onToggleFavorite;


  const CatalogScreen({
    super.key,
    required this.favoriteIds,
    required this.onToggleFavorite,
  });


  @override
  Widget build(BuildContext context) {
    final courses = CourseItem.sampleCourses;


    return Scaffold(
      backgroundColor: AppColors.background,
      appBar: AppBar(
        title: const Text('Каталог курсів'),
        centerTitle: true,
        backgroundColor: AppColors.surface,
        elevation: 0.5,
      ),
      body: SafeArea(
        child: Column(
          children: [
            // Інформаційна плашка про стан обраного
            Container(
              padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
              color: AppColors.surface,
              child: Row(
                children: [
                  const Icon(
                    Icons.storage_outlined,
                    size: 20,
                    color: AppColors.primary,
                  ),
                  const SizedBox(width: 8),
                  Expanded(
                    child: Text(
                      'Всього курсів: ${courses.length} | В обраному: ${favoriteIds.length}',
                      style: const TextStyle(
                        fontSize: 13,
                        fontWeight: FontWeight.w600,
                        color: AppColors.textPrimary,
                      ),
                    ),
                  ),
                  Container(
                    padding: const EdgeInsets.symmetric(
                      horizontal: 8,
                      vertical: 4,
                    ),
                    decoration: BoxDecoration(
                      color: AppColors.primary.withValues(alpha: 0.1),
                      borderRadius: BorderRadius.circular(12),
                    ),
                    child: const Text(
                      'SharedPreferences',
```

```
                style: TextStyle(
                  fontSize: 11,
                  fontWeight: FontWeight.bold,
                  color: AppColors.primary,
                ),
              ),
            ),
          ],
        ),
      ),
      const Divider(height: 1, thickness: 1, color: AppColors.border),


      // Список карток курсів
      Expanded(
        child: ListView.builder(
          padding: const EdgeInsets.all(12),
          itemCount: courses.length,
          itemBuilder: (context, index) {
            final course = courses[index];
            final isFav = favoriteIds.contains(course.id);


            return Card(
              key: Key('course_card_${course.id}'),
              margin: const EdgeInsets.only(bottom: 12),
              elevation: 1.5,
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(16),
                side: BorderSide(
                  color: isFav
                      ? AppColors.favorite.withValues(alpha: 0.4)
                      : AppColors.border,
                  width: isFav ? 1.5 : 1.0,
                ),
              ),
              color: AppColors.surface,
              child: Padding(
                padding: const EdgeInsets.all(14),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Row(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        // Іконка курсу
                        Container(
                          width: 48,
                          height: 48,
                          decoration: BoxDecoration(
                            color: course.accentColor.withValues(
                              alpha: 0.12,
                            ),
                            borderRadius: BorderRadius.circular(12),
                          ),
                          child: Icon(
                            course.icon,
                            color: course.accentColor,
                            size: 26,
                          ),
                        ),
                        const SizedBox(width: 12),
                        // Назва та категорія
                        Expanded(
```

```
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Container(
                      padding: const EdgeInsets.symmetric(
                        horizontal: 8,
                        vertical: 2,
                      ),
                      decoration: BoxDecoration(
                        color: course.accentColor.withValues(
                          alpha: 0.1,
                        ),
                        borderRadius: BorderRadius.circular(6),
                      ),
                      child: Text(
                        course.category,
                        style: TextStyle(
                          fontSize: 11,
                          fontWeight: FontWeight.bold,
                          color: course.accentColor,
                        ),
                      ),
                    ),
                    const SizedBox(height: 4),
                    Text(
                      course.title,
                      style: const TextStyle(
                        fontSize: 15,
                        fontWeight: FontWeight.bold,
                        color: AppColors.textPrimary,
                      ),
                    ),
                  ],
                ),
              ),
              // Кнопка перемикання обраного (п. 6 ТЗ)
              IconButton(
                key: Key('fav_btn_${course.id}'),
                icon: AnimatedSwitcher(
                  duration: const Duration(milliseconds: 250),
                  child: Icon(
                    isFav
                        ? Icons.favorite
                        : Icons.favorite_border,
                    key: ValueKey(isFav),
                    color: isFav
                        ? AppColors.favorite
                        : AppColors.textSecondary,
                    size: 26,
                  ),
                ),
                tooltip: isFav
                    ? 'Вилучити з обраного'
                    : 'Додати до обраного',
                onPressed: () {
                  onToggleFavorite(course.id);
                  ScaffoldMessenger.of(context).clearSnackBars();
                  ScaffoldMessenger.of(context).showSnackBar(
                    SnackBar(
                      content: Text(
                        isFav
                            ? '«${course.title}» вилучено з обраного'
                            : '«${course.title}» додано до обраного!',
                      ),
```

```
                backgroundColor: isFav
                    ? AppColors.textPrimary
                    : AppColors.favorite,
                duration: const Duration(seconds: 1),
              ),
            );
          },
        ),
      ],
    ),
    const SizedBox(height: 8),
    Text(
      course.description,
      style: const TextStyle(
        fontSize: 13,
        color: AppColors.textSecondary,
        height: 1.35,
      ),
    ),
    const SizedBox(height: 12),
    const Divider(height: 1, color: AppColors.border),
    const SizedBox(height: 8),
    // Статистика курсу (рейтинг, студенти, тривалість)
    Row(
      children: [
        const Icon(
          Icons.star_rounded,
          size: 18,
          color: Colors.amber,
        ),
        const SizedBox(width: 4),
        Text(
          course.rating.toString(),
          style: const TextStyle(
            fontSize: 13,
            fontWeight: FontWeight.bold,
          ),
        ),
        const SizedBox(width: 14),
        const Icon(
          Icons.people_outline,
          size: 16,
          color: AppColors.textSecondary,
        ),
        const SizedBox(width: 4),
        Text(
          '${course.studentsCount}',
          style: const TextStyle(
            fontSize: 12,
            color: AppColors.textSecondary,
          ),
        ),
        const Spacer(),
        const Icon(
          Icons.access_time,
          size: 16,
          color: AppColors.textSecondary,
        ),
        const SizedBox(width: 4),
        Text(
          course.duration,
          style: const TextStyle(
            fontSize: 12,
            fontWeight: FontWeight.w500,
```

```
                                color: AppColors.textSecondary,
                              ),
                            ),
                          ],
                        ),
                      ],
                    ),
                  );
                },
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

1.8. Екран обраних елементів (lib/screens/favorites_screen.dart):

```
import 'package:flutter/material.dart';
import '../constants/app_colors.dart';
import '../models/course_item.dart';


/// Екран списку обраних курсів (п. 8, 9 ТЗ)
/// Відображає виключно ті елементи, id яких збережені у SharedPreferences
class FavoritesScreen extends StatelessWidget {
  final List<String> favoriteIds;
  final ValueChanged<String> onToggleFavorite;
  final VoidCallback onClearAllFavorites;
  final VoidCallback onNavigateToCatalog;


  const FavoritesScreen({
    super.key,
    required this.favoriteIds,
    required this.onToggleFavorite,
    required this.onClearAllFavorites,
    required this.onNavigateToCatalog,
  });


  @override
  Widget build(BuildContext context) {
    // Фільтрація: відображаємо лише ті елементи, id яких містяться у favoriteIds (п. 9 ТЗ)
    final favoriteCourses = CourseItem.sampleCourses
        .where((c) => favoriteIds.contains(c.id))
        .toList();


    return Scaffold(
      backgroundColor: AppColors.background,
      appBar: AppBar(
        title: const Text('Обрані курси'),
        centerTitle: true,
        backgroundColor: AppColors.surface,
        elevation: 0.5,
        actions: [
          if (favoriteCourses.isNotEmpty)
            IconButton(
```

```
              icon: const Icon(Icons.delete_sweep_outlined),
              tooltip: 'Очистити все обране',
              onPressed: () => _confirmClearFavorites(context),
            ),
          ],
        ),
        body: SafeArea(
          child: favoriteCourses.isEmpty
              ? _buildEmptyState(context)
              : _buildFavoritesList(context, favoriteCourses),
        ),
      );
    }

    /// Відображення порожнього стану, коли в SharedPreferences немає збережених id
    Widget _buildEmptyState(BuildContext context) {
      return Center(
        child: Padding(
          padding: const EdgeInsets.all(32.0),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Container(
                padding: const EdgeInsets.all(24),
                decoration: BoxDecoration(
                  color: AppColors.favorite.withValues(alpha: 0.1),
                  shape: BoxShape.circle,
                ),
                child: const Icon(
                  Icons.favorite_border_rounded,
                  size: 64,
                  color: AppColors.favorite,
                ),
              ),
              const SizedBox(height: 20),
              const Text(
                'Список обраного порожній',
                textAlign: TextAlign.center,
                style: TextStyle(
                  fontSize: 20,
                  fontWeight: FontWeight.bold,
                  color: AppColors.textPrimary,
                ),
              ),
              const SizedBox(height: 8),
              const Text(
                'Ви ще не додали жодного курсу. Перейдіть до каталогу та позначте серцем те, що
вас цікавить.',
                textAlign: TextAlign.center,
                style: TextStyle(
                  fontSize: 14,
                  color: AppColors.textSecondary,
                  height: 1.4,
                ),
              ),
              const SizedBox(height: 24),
              ElevatedButton.icon(
                style: ElevatedButton.styleFrom(
                  backgroundColor: AppColors.primary,
                  foregroundColor: Colors.white,
                  padding: const EdgeInsets.symmetric(
                    horizontal: 24,
                    vertical: 13,
```

```
              ),
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(12),
              ),
            ),
            icon: const Icon(Icons.search),
            label: const Text('Перейти до каталогу'),
            onPressed: onNavigateToCatalog,
          ),
        ],
      ),
    ),
  );
}


/// Список збережених елементів
Widget _buildFavoritesList(
  BuildContext context,
  List<CourseItem> favoriteCourses,
) {
  return Column(
    children: [
      // Інформаційний банер
      Container(
        padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
        color: AppColors.surface,
        child: Row(
          children: [
            const Icon(Icons.favorite, size: 20, color: AppColors.favorite),
            const SizedBox(width: 8),
            Expanded(
              child: Text(
                'Збережено елементів у SharedPreferences: ${favoriteCourses.length}',
                style: const TextStyle(
                  fontSize: 13,
                  fontWeight: FontWeight.w600,
                  color: AppColors.textPrimary,
                ),
              ),
            ),
            Text(
              'IDs: ${favoriteIds.join(", ")}',
              style: const TextStyle(
                fontSize: 11,
                fontFamily: 'monospace',
                color: AppColors.textSecondary,
              ),
            ),
          ],
        ),
      ),
      const Divider(height: 1, thickness: 1, color: AppColors.border),


      // Список карток обраного
      Expanded(
        child: ListView.builder(
          padding: const EdgeInsets.all(12),
          itemCount: favoriteCourses.length,
          itemBuilder: (context, index) {
            final course = favoriteCourses[index];


            return Card(
```

```
                key: Key('fav_card_${course.id}'),
                margin: const EdgeInsets.only(bottom: 12),
                elevation: 1.5,
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(16),
                  side: const BorderSide(
                    color: AppColors.border,
                    width: 1.0,
                  ),
                ),
                color: AppColors.surface,
                child: Padding(
                  padding: const EdgeInsets.all(14),
                  child: Row(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      // Іконка курсу
                      Container(
                        width: 44,
                        height: 44,
                        decoration: BoxDecoration(
                          color: course.accentColor.withValues(alpha: 0.12),
                          borderRadius: BorderRadius.circular(12),
                        ),
                        child: Icon(
                          course.icon,
                          color: course.accentColor,
                          size: 24,
                        ),
                      ),
                      const SizedBox(width: 12),
                      // Опис
                      Expanded(
                        child: Column(
                          crossAxisAlignment: CrossAxisAlignment.start,
                          children: [
                            Text(
                              course.title,
                              style: const TextStyle(
                                fontSize: 14,
                                fontWeight: FontWeight.bold,
                                color: AppColors.textPrimary,
                              ),
                            ),
                            const SizedBox(height: 4),
                            Text(
                              course.category,
                              style: TextStyle(
                                fontSize: 12,
                                fontWeight: FontWeight.w600,
                                color: course.accentColor,
                              ),
                            ),
                            const SizedBox(height: 6),
                            Row(
                              children: [
                                const Icon(
                                  Icons.star_rounded,
                                  size: 16,
                                  color: Colors.amber,
                                ),
                                const SizedBox(width: 3),
                                Text(
                                  '${course.rating}',
```

```
                                style: const TextStyle(
                                  fontSize: 12,
                                  fontWeight: FontWeight.bold,
                                ),
                              ),
                              const SizedBox(width: 12),
                              Text(
                                course.duration,
                                style: const TextStyle(
                                  fontSize: 12,
                                  color: AppColors.textSecondary,
                                ),
                              ),
                            ],
                          ),
                        ],
                      ),
                    ),
                    // Кнопка видалення з обраного
                    IconButton(
                      key: Key('remove_fav_${course.id}'),
                      icon: const Icon(
                        Icons.favorite,
                        color: AppColors.favorite,
                        size: 26,
                      ),
                      tooltip: 'Вилучити з обраного',
                      onPressed: () {
                        onToggleFavorite(course.id);
                        ScaffoldMessenger.of(context).clearSnackBars();
                        ScaffoldMessenger.of(context).showSnackBar(
                          SnackBar(
                            content: Text(
                              '«${course.title}» вилучено з обраного',
                            ),
                            duration: const Duration(seconds: 1),
                          ),
                        );
                      },
                    ),
                  ],
                ),
              ),
            );
          },
        ),
      ),
    ],
  );
}

void _confirmClearFavorites(BuildContext context) {
  showDialog(
    context: context,
    builder: (ctx) => AlertDialog(
      title: const Text('Очистити список обраного?'),
      content: const Text(
        'Усі збережені ідентифікатори курсів будуть видалені з SharedPreferences.',
      ),
      actions: [
        TextButton(
          child: const Text('Скасувати'),
          onPressed: () => Navigator.of(ctx).pop(),
```

```
          ),
          ElevatedButton(
            style: ElevatedButton.styleFrom(
              backgroundColor: AppColors.error,
              foregroundColor: Colors.white,
            ),
            child: const Text('Очистити'),
            onPressed: () {
              Navigator.of(ctx).pop();
              onClearAllFavorites();
              ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(
                  content: Text('Список обраного повністю очищено'),
                ),
              );
            },
          ),
        ],
      ),
    );
  }
}
```

1.9. Екран профілю користувача (lib/screens/profile_screen.dart):

```
import 'package:flutter/material.dart';
import '../constants/app_colors.dart';
import '../models/user_model.dart';


/// Екран профілю користувача (п. 4, 5 ТЗ)
/// Відображає дані, завантажені з SharedPreferences: ім'я, email, пароль, контакти
class ProfileScreen extends StatefulWidget {
  final UserModel user;
  final int favoritesCount;
  final VoidCallback onEditProfile;
  final VoidCallback onLogout;


  const ProfileScreen({
    super.key,
    required this.user,
    required this.favoritesCount,
    required this.onEditProfile,
    required this.onLogout,
  });


  @override
  State<ProfileScreen> createState() => _ProfileScreenState();
}


class _ProfileScreenState extends State<ProfileScreen> {
  bool _showPassword = false;


  @override
  Widget build(BuildContext context) {
    final user = widget.user;
```

```
    return Scaffold(
      backgroundColor: AppColors.background,
      appBar: AppBar(
        title: const Text('Профіль користувача'),
        centerTitle: true,
        backgroundColor: AppColors.surface,
        elevation: 0.5,
        actions: [
          IconButton(
            key: const Key('edit_profile_btn'),
            icon: const Icon(Icons.edit_outlined),
            tooltip: 'Редагувати профіль',
            onPressed: widget.onEditProfile,
          ),
          IconButton(
            key: const Key('logout_btn'),
            icon: const Icon(Icons.logout),
            tooltip: 'Вийти з акаунта',
            onPressed: () => _confirmLogout(context),
          ),
        ],
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(16.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              // Картка аватара та основного імені
              Container(
                padding: const EdgeInsets.symmetric(
                  vertical: 20,
                  horizontal: 16,
                ),
                decoration: BoxDecoration(
                  color: AppColors.surface,
                  borderRadius: BorderRadius.circular(20),
                  border: Border.all(color: AppColors.border),
                  boxShadow: [
                    BoxShadow(
                      color: Colors.black.withValues(alpha: 0.04),
                      blurRadius: 10,
                      offset: const Offset(0, 4),
                    ),
                  ],
                ),
                child: Column(
                  children: [
                    Stack(
                      alignment: Alignment.bottomRight,
                      children: [
                        CircleAvatar(
                          radius: 46,
                          backgroundColor:
                              AppColors.primary.withValues(alpha: 0.15),
                          child: Text(
                            user.name.isNotEmpty
                                ? user.name
                                      .trim()
                                      .split(' ')
                                      .map((e) => e[0])
                                      .take(2)
                                      .join('')
                                : 'U',
```

```
                        style: const TextStyle(
                          fontSize: 32,
                          fontWeight: FontWeight.bold,
                          color: AppColors.primary,
                        ),
                      ),
                    ),
                    Container(
                      width: 18,
                      height: 18,
                      decoration: BoxDecoration(
                        color: AppColors.success,
                        shape: BoxShape.circle,
                        border: Border.all(color: Colors.white, width: 3),
                      ),
                    ),
                  ],
                ),
                const SizedBox(height: 14),
                // Ім'я користувача (п. 3.a, 5 ТЗ)
                Text(
                  user.name.isNotEmpty ? user.name : 'Гість',
                  key: const Key('profile_name_text'),
                  textAlign: TextAlign.center,
                  style: const TextStyle(
                    fontSize: 22,
                    fontWeight: FontWeight.bold,
                    color: AppColors.textPrimary,
                  ),
                ),
                const SizedBox(height: 4),
                // Email користувача (п. 3.b, 5 ТЗ)
                Row(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    const Icon(
                      Icons.email_outlined,
                      size: 16,
                      color: AppColors.primary,
                    ),
                    const SizedBox(width: 6),
                    Text(
                      user.email.isNotEmpty ? user.email : 'не вказано',
                      key: const Key('profile_email_text'),
                      style: const TextStyle(
                        fontSize: 14,
                        fontWeight: FontWeight.w500,
                        color: AppColors.textSecondary,
                      ),
                    ),
                  ],
                ),
                const SizedBox(height: 10),
                // Статус / Опис
                Container(
                  padding: const EdgeInsets.symmetric(
                    horizontal: 14,
                    vertical: 8,
                  ),
                  decoration: BoxDecoration(
                    color: AppColors.background,
                    borderRadius: BorderRadius.circular(10),
                  ),
                  child: Text(
```

```
                user.bio,
                textAlign: TextAlign.center,
                style: const TextStyle(
                  fontSize: 13,
                  color: AppColors.textPrimary,
                ),
              ),
            ),
          ],
        ),
      ),

      const SizedBox(height: 16),

      // Блок статистики та джерела збереження
      Row(
        children: [
          Expanded(
            child: _buildInfoCard(
              icon: Icons.favorite,
              iconColor: AppColors.favorite,
              title: 'В обраному',
              value: '${widget.favoritesCount}',
            ),
          ),
          const SizedBox(width: 12),
          Expanded(
            child: _buildInfoCard(
              icon: Icons.storage_rounded,
              iconColor: AppColors.primary,
              title: 'Сховище',
              value: 'SharedPrefs',
            ),
          ),
          const SizedBox(width: 12),
          Expanded(
            child: _buildInfoCard(
              icon: Icons.check_circle_outline,
              iconColor: AppColors.success,
              title: 'Статус',
              value: 'Активний',
            ),
          ),
        ],
      ),

      const SizedBox(height: 16),

      // Детальні дані з форми реєстрації (п. 3, 5 ТЗ)
      Card(
        elevation: 1,
        color: AppColors.surface,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(16),
          side: const BorderSide(color: AppColors.border),
        ),
        child: Padding(
          padding: const EdgeInsets.symmetric(
            horizontal: 16,
            vertical: 8,
          ),
```

```
                child: Column(
                  children: [
                    // Телефон
                    ListTile(
                      contentPadding: EdgeInsets.zero,
                      leading: const Icon(
                        Icons.phone_android_outlined,
                        color: AppColors.primary,
                      ),
                      title: const Text(
                        'Номер телефону',
                        style: TextStyle(
                          fontSize: 12,
                          color: AppColors.textSecondary,
                        ),
                      ),
                      subtitle: Text(
                        user.phone.isNotEmpty ? user.phone : 'не вказано',
                        style: const TextStyle(
                          fontSize: 15,
                          fontWeight: FontWeight.w600,
                          color: AppColors.textPrimary,
                        ),
                      ),
                    ),
                    const Divider(height: 1, color: AppColors.border),


                    // Пароль зі збереження (п. 3.c ТЗ)
                    ListTile(
                      contentPadding: EdgeInsets.zero,
                      leading: const Icon(
                        Icons.lock_outline,
                        color: AppColors.primary,
                      ),
                      title: const Text(
                        'Пароль (збережений у SharedPreferences)',
                        style: TextStyle(
                          fontSize: 12,
                          color: AppColors.textSecondary,
                        ),
                      ),
                      subtitle: Text(
                        _showPassword
                            ? user.password
                            : '•' * (user.password.length.clamp(6, 12)),
                        key: const Key('profile_password_text'),
                        style: const TextStyle(
                          fontSize: 15,
                          fontWeight: FontWeight.bold,
                          letterSpacing: 1.2,
                          color: AppColors.textPrimary,
                        ),
                      ),
                      trailing: IconButton(
                        key: const Key('toggle_password_btn'),
                        icon: Icon(
                          _showPassword
                              ? Icons.visibility_off
                              : Icons.visibility,
                          color: AppColors.textSecondary,
                        ),
                        onPressed: () {
                          setState(() => _showPassword = !_showPassword);
```

```
                    },
                  ),
                ),
                const Divider(height: 1, color: AppColors.border),

                // Дата збереження
                ListTile(
                  contentPadding: EdgeInsets.zero,
                  leading: const Icon(
                    Icons.calendar_today_outlined,
                    color: AppColors.primary,
                  ),
                  title: const Text(
                    'Дата реєстрації / збереження',
                    style: TextStyle(
                      fontSize: 12,
                      color: AppColors.textSecondary,
                    ),
                  ),
                  subtitle: Text(
                    user.registeredAt.isNotEmpty
                        ? user.registeredAt
                        : 'Щойно',
                    style: const TextStyle(
                      fontSize: 14,
                      fontWeight: FontWeight.w500,
                      color: AppColors.textPrimary,
                    ),
                  ),
                ),
              ],
            ),
          ),
        ),

        const SizedBox(height: 20),

        // Дії користувача
        ElevatedButton.icon(
          style: ElevatedButton.styleFrom(
            backgroundColor: AppColors.primary,
            foregroundColor: Colors.white,
            padding: const EdgeInsets.symmetric(vertical: 14),
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(12),
            ),
          ),
          icon: const Icon(Icons.edit, size: 20),
          label: const Text(
            'Оновити дані реєстрації',
            style: TextStyle(fontSize: 15, fontWeight: FontWeight.bold),
          ),
          onPressed: widget.onEditProfile,
        ),

        const SizedBox(height: 10),

        OutlinedButton.icon(
          key: const Key('logout_outline_btn'),
          style: OutlinedButton.styleFrom(
            foregroundColor: AppColors.error,
```

```
                side: const BorderSide(color: AppColors.error),
                padding: const EdgeInsets.symmetric(vertical: 14),
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(12),
                ),
              ),
              icon: const Icon(Icons.delete_outline, size: 20),
              label: const Text(
                'Вийти та очистити SharedPreferences',
                style: TextStyle(fontSize: 15, fontWeight: FontWeight.bold),
              ),
              onPressed: () => _confirmLogout(context),
            ),
          ],
        ),
      ),
    ),
  );
}

Widget _buildInfoCard({
  required IconData icon,
  required Color iconColor,
  required String title,
  required String value,
}) {
  return Container(
    padding: const EdgeInsets.symmetric(vertical: 14, horizontal: 8),
    decoration: BoxDecoration(
      color: AppColors.surface,
      borderRadius: BorderRadius.circular(14),
      border: Border.all(color: AppColors.border),
    ),
    child: Column(
      children: [
        Icon(icon, color: iconColor, size: 22),
        const SizedBox(height: 6),
        Text(
          value,
          style: const TextStyle(
            fontSize: 15,
            fontWeight: FontWeight.bold,
            color: AppColors.textPrimary,
          ),
        ),
        const SizedBox(height: 2),
        Text(
          title,
          style: const TextStyle(
            fontSize: 11,
            color: AppColors.textSecondary,
          ),
        ),
      ],
    ),
  );
}

void _confirmLogout(BuildContext context) {
  showDialog(
    context: context,
    builder: (ctx) => AlertDialog(
```

```
        title: const Text('Вихід та видалення даних'),
        content: const Text(
          'Всі збережені дані профілю будуть видалені з SharedPreferences, і застосунок
повернеться до екрана реєстрації.',
        ),
        actions: [
          TextButton(
            child: const Text('Скасувати'),
            onPressed: () => Navigator.of(ctx).pop(),
          ),
          ElevatedButton(
            style: ElevatedButton.styleFrom(
              backgroundColor: AppColors.error,
              foregroundColor: Colors.white,
            ),
            child: const Text('Вийти та видалити'),
            onPressed: () {
              Navigator.of(ctx).pop();
              widget.onLogout();
            },
          ),
        ],
      ),
    );
  }
}
```

1.10. Головна навігація застосунку (lib/screens/main_navigation_screen.dart):

```
import 'package:flutter/material.dart';
import '../constants/app_colors.dart';
import '../models/user_model.dart';
import '../services/storage_service.dart';
import 'catalog_screen.dart';
import 'favorites_screen.dart';
import 'profile_screen.dart';
import 'registration_screen.dart';


/// Головний навігаційний екран застосунку з BottomNavigationBar (п. 4, 8, 9 ТЗ)
class MainNavigationScreen extends StatefulWidget {
  final StorageService storageService;
  final UserModel initialUser;
  final List<String> initialFavoriteIds;
  final VoidCallback onLogout;


  const MainNavigationScreen({
    super.key,
    required this.storageService,
    required this.initialUser,
    required this.initialFavoriteIds,
    required this.onLogout,
  });


  @override
  State<MainNavigationScreen> createState() => _MainNavigationScreenState();
}


class _MainNavigationScreenState extends State<MainNavigationScreen> {
```

```
int _currentIndex = 0;
late UserModel _currentUser;
late List<String> _favoriteIds;


@override
void initState() {
  super.initState();
  _currentUser = widget.initialUser;
  _favoriteIds = List<String>.from(widget.initialFavoriteIds);
}


/// Додавання / вилучення курсу з обраного (п. 6, 7 ТЗ)
Future<void> _handleToggleFavorite(String id) async {
  await widget.storageService.toggleFavorite(id);
  final updated = await widget.storageService.loadFavoriteIds();
  setState(() {
    _favoriteIds = updated;
  });
}


/// Повне очищення списку обраного
Future<void> _handleClearAllFavorites() async {
  await widget.storageService.clearFavorites();
  setState(() {
    _favoriteIds = [];
  });
}


/// Перехід у режим редагування профілю
void _handleEditProfile() {
  Navigator.of(context).push(
    MaterialPageRoute(
      builder: (ctx) => RegistrationScreen(
        storageService: widget.storageService,
        initialUser: _currentUser,
        onRegistered: () async {
          final reloaded = await widget.storageService.loadUser();
          if (reloaded != null) {
            setState(() {
              _currentUser = reloaded;
            });
          }
          if (ctx.mounted) {
            Navigator.of(ctx).pop();
          }
        },
      ),
    ),
  );
}


@override
Widget build(BuildContext context) {
  final screens = [
    // Вкладка 0: Каталог курсів
    CatalogScreen(
      favoriteIds: _favoriteIds,
      onToggleFavorite: _handleToggleFavorite,
    ),
```

```
      // Вкладка 1: Обране (п. 8, 9 ТЗ)
      FavoritesScreen(
        favoriteIds: _favoriteIds,
        onToggleFavorite: _handleToggleFavorite,
        onClearAllFavorites: _handleClearAllFavorites,
        onNavigateToCatalog: () {
          setState(() => _currentIndex = 0);
        },
      ),


      // Вкладка 2: Профіль користувача (п. 5 ТЗ)
      ProfileScreen(
        user: _currentUser,
        favoritesCount: _favoriteIds.length,
        onEditProfile: _handleEditProfile,
        onLogout: widget.onLogout,
      ),
    ];


    return Scaffold(
      body: IndexedStack(
        index: _currentIndex,
        children: screens,
      ),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _currentIndex,
        onDestinationSelected: (index) {
          setState(() => _currentIndex = index);
        },
        indicatorColor: AppColors.primary.withValues(alpha: 0.15),
        destinations: [
          const NavigationDestination(
            key: Key('nav_catalog'),
            icon: Icon(Icons.school_outlined),
            selectedIcon: Icon(Icons.school, color: AppColors.primary),
            label: 'Каталог',
          ),
          NavigationDestination(
            key: const Key('nav_favorites'),
            icon: Badge(
              isLabelVisible: _favoriteIds.isNotEmpty,
              label: Text('${_favoriteIds.length}'),
              child: const Icon(Icons.favorite_border),
            ),
            selectedIcon: Badge(
              isLabelVisible: _favoriteIds.isNotEmpty,
              label: Text('${_favoriteIds.length}'),
              child: const Icon(Icons.favorite, color: AppColors.favorite),
            ),
            label: 'Обране',
          ),
          NavigationDestination(
            key: const Key('nav_profile'),
            icon: const Icon(Icons.person_outline),
            selectedIcon: const Icon(Icons.person, color: AppColors.primary),
            label: 'Профіль',
          ),
        ],
      ),
    );
  }
}
```

1.11. Точка входу та bootstrap відновлення даних (lib/main.dart):

```
import 'package:flutter/material.dart';
import 'constants/app_colors.dart';
import 'models/user_model.dart';
import 'screens/main_navigation_screen.dart';
import 'screens/registration_screen.dart';
import 'services/storage_service.dart';


void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const StorageApp());
}


/// Головний віджет застосунку (п. 4, 8, 10 ТЗ)
class StorageApp extends StatefulWidget {
  final StorageService? storageService;


  const StorageApp({super.key, this.storageService});


  @override
  State<StorageApp> createState() => _StorageAppState();
}


class _StorageAppState extends State<StorageApp> {
  late final StorageService _storageService;


  bool _isLoading = true;
  UserModel? _currentUser;
  List<String> _favoriteIds = [];


  @override
  void initState() {
    super.initState();
    _storageService = widget.storageService ?? StorageService();
    _bootstrap();
  }


  /// Завантаження даних користувача та favorites ids після запуску (п. 4, 8 ТЗ)
  Future<void> _bootstrap() async {
    setState(() => _isLoading = true);


    final user = await _storageService.loadUser();
    final favIds = await _storageService.loadFavoriteIds();


    if (!mounted) return;
    setState(() {
      _currentUser = user;
      _favoriteIds = favIds;
      _isLoading = false;
    });
  }


  /// Обробник успішної реєстрації
```

```
  Future<void> _handleRegistrationSuccess() async {
    final user = await _storageService.loadUser();
    final favIds = await _storageService.loadFavoriteIds();
    setState(() {
      _currentUser = user;
      _favoriteIds = favIds;
    });
  }


  /// Обробник виходу з профілю
  Future<void> _handleLogout() async {
    await _storageService.clearUser();
    setState(() {
      _currentUser = null;
    });
  }


  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Локальне збереження даних — Лабораторна №10',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: AppColors.primary,
          primary: AppColors.primary,
          secondary: AppColors.secondary,
          surface: AppColors.surface,
        ),
        scaffoldBackgroundColor: AppColors.background,
        appBarTheme: const AppBarTheme(
          backgroundColor: AppColors.surface,
          foregroundColor: AppColors.textPrimary,
          elevation: 0.5,
          centerTitle: true,
        ),
      ),
      home: _isLoading
          ? const Scaffold(
              body: Center(
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    CircularProgressIndicator(),
                    SizedBox(height: 16),
                    Text(
                      'Завантаження даних із SharedPreferences...',
                      style: TextStyle(color: AppColors.textSecondary),
                    ),
                  ],
                ),
              ),
            )
          : _currentUser == null
              ? RegistrationScreen(
                  storageService: _storageService,
                  onRegistered: _handleRegistrationSuccess,
                )
              : MainNavigationScreen(
                  storageService: _storageService,
                  initialUser: _currentUser!,
```

```
                initialFavoriteIds: _favoriteIds,
                onLogout: _handleLogout,
              ),
    );
  }
}
```

1.12. Автоматизовані модульні та віджет-тести (test/widget_test.dart):

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:storage_app/main.dart';
import 'package:storage_app/models/course_item.dart';
import 'package:storage_app/models/user_model.dart';
import 'package:storage_app/services/storage_service.dart';


void main() {
  setUp(() {
    SharedPreferences.setMockInitialValues({});
  });


  group('StorageService Unit Tests (п. 1-8 ТЗ)', () {
    test('Збереження та завантаження даних користувача', () async {
      SharedPreferences.setMockInitialValues({});
      final service = StorageService();


      expect(await service.loadUser(), isNull);
      expect(await service.isUserRegistered(), isFalse);


      const user = UserModel(
        name: 'Багрій-Белз Андрій',
        email: 'andrii.bagriy@nulp.ua',
        password: 'SuperPassword123',
        phone: '+380 67 123 45 67',
        bio: 'Студент групи КН-2327Б',
        registeredAt: '2026-10-06 14:00:00',
      );


      await service.saveUser(user);


      expect(await service.isUserRegistered(), isTrue);
      final loaded = await service.loadUser();
      expect(loaded, isNotNull);
      expect(loaded!.name, 'Багрій-Белз Андрій');
      expect(loaded.email, 'andrii.bagriy@nulp.ua');
      expect(loaded.password, 'SuperPassword123');
      expect(loaded.phone, '+380 67 123 45 67');
      expect(loaded.bio, 'Студент групи КН-2327Б');
    });


    test('Збереження та перемикання favorites ids (п. 6, 7 ТЗ)', () async {
      SharedPreferences.setMockInitialValues({});
      final service = StorageService();
```

```
    expect(await service.loadFavoriteIds(), isEmpty);

    // Додавання course_1
    final added1 = await service.toggleFavorite('course_1');
    expect(added1, isTrue);
    expect(await service.loadFavoriteIds(), ['course_1']);
    expect(await service.isFavorite('course_1'), isTrue);

    // Додавання course_2
    final added2 = await service.toggleFavorite('course_2');
    expect(added2, isTrue);
    expect(await service.loadFavoriteIds(), ['course_1', 'course_2']);

    // Вилучення course_1
    final removed1 = await service.toggleFavorite('course_1');
    expect(removed1, isFalse);
    expect(await service.loadFavoriteIds(), ['course_2']);
    expect(await service.isFavorite('course_1'), isFalse);
    expect(await service.isFavorite('course_2'), isTrue);

    // Очищення
    await service.clearFavorites();
    expect(await service.loadFavoriteIds(), isEmpty);
  });

  test('Очищення облікових даних користувача (clearUser)', () async {
    SharedPreferences.setMockInitialValues({});
    final service = StorageService();

    const user = UserModel(
      name: 'Тестовий Користувач',
      email: 'test@example.com',
      password: 'pass',
    );
    await service.saveUser(user);
    expect(await service.isUserRegistered(), isTrue);

    await service.clearUser();
    expect(await service.isUserRegistered(), isFalse);
    expect(await service.loadUser(), isNull);
  });
});

group('Віджет-тести UI та інтеграції (п. 2, 4, 5, 8, 9, 10 ТЗ)', () {
  testWidgets('Початковий запуск без даних відкриває екран реєстрації', (
    tester,
  ) async {
    SharedPreferences.setMockInitialValues({});
    await tester.pumpWidget(const StorageApp());
    await tester.pumpAndSettle();

    expect(find.text('Реєстрація користувача'), findsOneWidget);
    expect(find.byKey(const Key('name_field')), findsOneWidget);
    expect(find.byKey(const Key('email_field')), findsOneWidget);
    expect(find.byKey(const Key('password_field')), findsOneWidget);
    expect(find.byKey(const Key('register_button')), findsOneWidget);
  });
```

```
testWidgets(
  'Реєстрація користувача зберігає дані та веде до каталогу/профілю',
  (tester) async {
    SharedPreferences.setMockInitialValues({});
    await tester.pumpWidget(const StorageApp());
    await tester.pumpAndSettle();
    tester.view.physicalSize = const Size(1080, 2400);
    tester.view.devicePixelRatio = 2.0;
    addTearDown(() {
      tester.view.resetPhysicalSize();
      tester.view.resetDevicePixelRatio();
    });


    // Заповнення форми через кнопку демо-даних
    await tester.ensureVisible(find.byKey(const Key('demo_fill_button')));
    await tester.tap(find.byKey(const Key('demo_fill_button')));
    await tester.pumpAndSettle();


    expect(find.text('Багрій-Белз Андрій'), findsOneWidget);
    expect(find.text('andrii.bagriy@nulp.ua'), findsOneWidget);


    // Натискання кнопки реєстрації
    await tester.ensureVisible(find.byKey(const Key('register_button')));
    await tester.tap(find.byKey(const Key('register_button')));
    await tester.pumpAndSettle();


    // Повинен відобразитися головний екран з каталогом
    expect(find.text('Каталог курсів'), findsOneWidget);
    expect(find.byKey(const Key('nav_profile')), findsOneWidget);


    // Перехід на екран профілю
    await tester.tap(find.byKey(const Key('nav_profile')));
    await tester.pumpAndSettle();


    // Перевірка відображення збережених даних на ProfileScreen (п. 5 ТЗ)
    expect(find.byKey(const Key('profile_name_text')), findsOneWidget);
    expect(find.text('Багрій-Белз Андрій'), findsWidgets);
    expect(find.byKey(const Key('profile_email_text')), findsOneWidget);
    expect(find.text('andrii.bagriy@nulp.ua'), findsOneWidget);
  },
);


testWidgets('Додавання в обране та фільтрація на FavoritesScreen (п. 6, 8, 9 ТЗ)', (
  tester,
) async {
  // Початкові збережені дані
  SharedPreferences.setMockInitialValues({
    StorageService.keyIsRegistered: true,
    StorageService.keyName: 'Багрій-Белз Андрій',
    StorageService.keyEmail: 'andrii.bagriy@nulp.ua',
    StorageService.keyPassword: 'SecurePassword123',
    StorageService.keyFavoriteIds: <String>['course_1'],
  });


  await tester.pumpWidget(const StorageApp());
  await tester.pumpAndSettle();


  // Має завантажитися одразу головний екран
```

```
    expect(find.text('Каталог курсів'), findsOneWidget);


    // Перехід до вкладки "Обране"
    await tester.tap(find.byKey(const Key('nav_favorites')));
    await tester.pumpAndSettle();


    // На FavoritesScreen має відображатися ТІЛЬКИ course_1 (п. 9 ТЗ)
    expect(find.text('Обрані курси'), findsOneWidget);
    expect(
      find.text(CourseItem.sampleCourses[0].title),
      findsOneWidget,
    ); // course_1
    expect(
      find.text(CourseItem.sampleCourses[1].title),
      findsNothing,
    ); // course_2 не в обраному


    // Повернення до каталогу та додавання course_2
    await tester.tap(find.byKey(const Key('nav_catalog')));
    await tester.pumpAndSettle();


    await tester.tap(find.byKey(const Key('fav_btn_course_2')));
    await tester.pumpAndSettle();


    // Знову перехід до обраного
    await tester.tap(find.byKey(const Key('nav_favorites')));
    await tester.pumpAndSettle();


    // Тепер обидва курси присутні
    expect(find.text(CourseItem.sampleCourses[0].title), findsOneWidget);
    expect(find.text(CourseItem.sampleCourses[1].title), findsOneWidget);
  });


  testWidgets(
    'Перевірка збереження даних після перезапуску застосунку (п. 10 ТЗ)',
    (tester) async {
      // Симуляція стану після тривалої роботи
      SharedPreferences.setMockInitialValues({
        StorageService.keyIsRegistered: true,
        StorageService.keyName: 'Багрій-Белз Андрій',
        StorageService.keyEmail: 'andrii.bagriy@nulp.ua',
        StorageService.keyPassword: 'MySecretPassword99',
        StorageService.keyPhone: '+380 67 999 88 77',
        StorageService.keyBio: 'Студент кафедри АСУ НУ ЛП',
        StorageService.keyFavoriteIds: <String>['course_2', 'course_4'],
      });


      // "Перезапуск" застосунку: новий інстанс StorageApp
      await tester.pumpWidget(const StorageApp());
      await tester.pumpAndSettle();


      // 1. Перевірка збереження профілю
      await tester.tap(find.byKey(const Key('nav_profile')));
      await tester.pumpAndSettle();


      expect(find.text('Багрій-Белз Андрій'), findsWidgets);
      expect(find.text('andrii.bagriy@nulp.ua'), findsOneWidget);
      expect(find.text('+380 67 999 88 77'), findsOneWidget);
```

```
        expect(find.text('Студент кафедри АСУ НУ ЛП'), findsOneWidget);


        // 2. Перевірка збереження обраних елементів
        await tester.tap(find.byKey(const Key('nav_favorites')));
        await tester.pumpAndSettle();


        expect(find.text(CourseItem.sampleCourses[1].title), findsOneWidget); // course_2
        expect(find.text(CourseItem.sampleCourses[3].title), findsOneWidget); // course_4
        expect(find.text(CourseItem.sampleCourses[0].title), findsNothing); // course_1 немає
      },
    );
  });
}
```

2. Результат виконання коду

Працездатність розробленого рішення перевірено автоматизованим тестуванням за допомогою утиліти **flutter test**, а також інтерактивним тестуванням на емуляторі Android 16 (API 36).

Результати виконання автоматизованих тестів:

```
00:00 +0: loading test/widget_test.dart
00:00 +0: StorageService Unit Tests: Збереження та завантаження даних користувача
00:00 +1: StorageService Unit Tests: Збереження та перемикання favorites ids (п. 6, 7 ТЗ)
00:00 +2: StorageService Unit Tests: Очищення облікових даних користувача (clearUser)
00:00 +3: Віджет-тести: Початковий запуск без даних відкриває екран реєстрації
00:00 +4: Віджет-тести: Реєстрація користувача зберігає дані та веде до каталогу/профілю
00:00 +5: Віджет-тести: Додавання в обране та фільтрація на FavoritesScreen (п. 6, 8, 9 ТЗ)
00:01 +6: Віджет-тести: Перевірка збереження даних після перезапуску застосунку (п. 10 ТЗ)
00:01 +7: All tests passed!
```

Для наочного підтвердження виконання всіх вимог технічного завдання наведено скріншоти роботи мобільного застосунку в емуляторі Android:

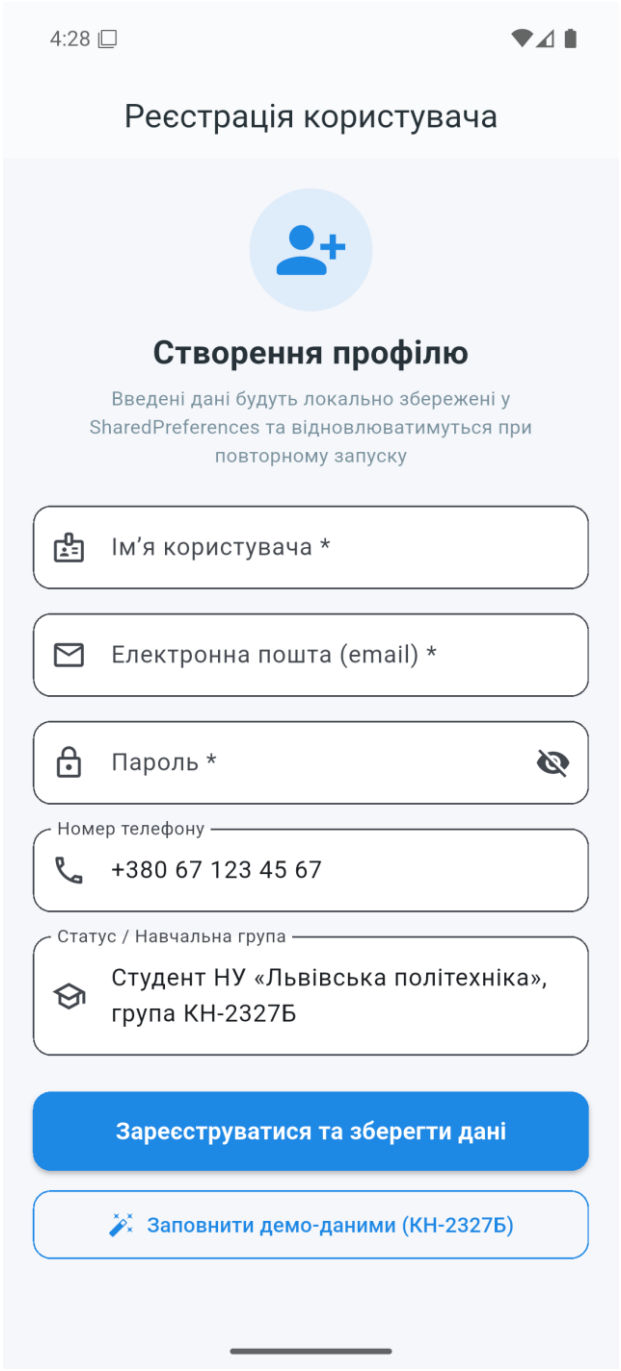


Рис. 1. Початковий запуск застосунку: екран реєстрації за відсутності локальних даних

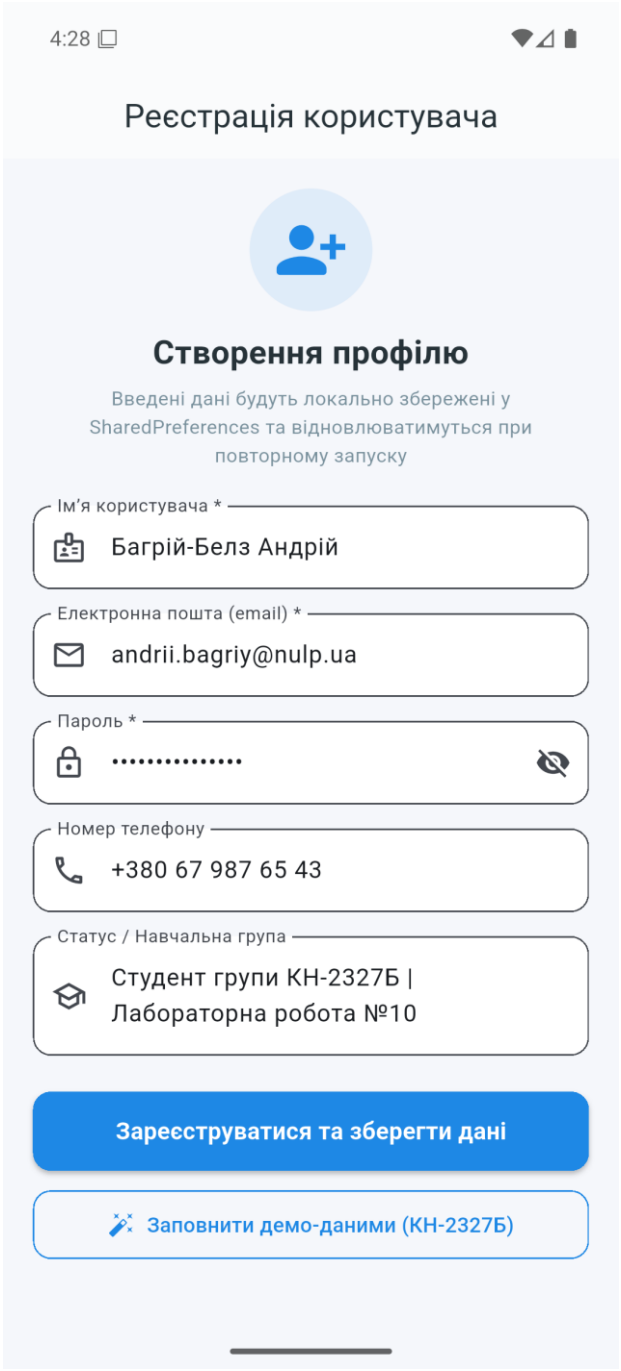


Рис. 2. Заповнення полів форми реєстрації (ім'я, email, пароль, телефон, статус)

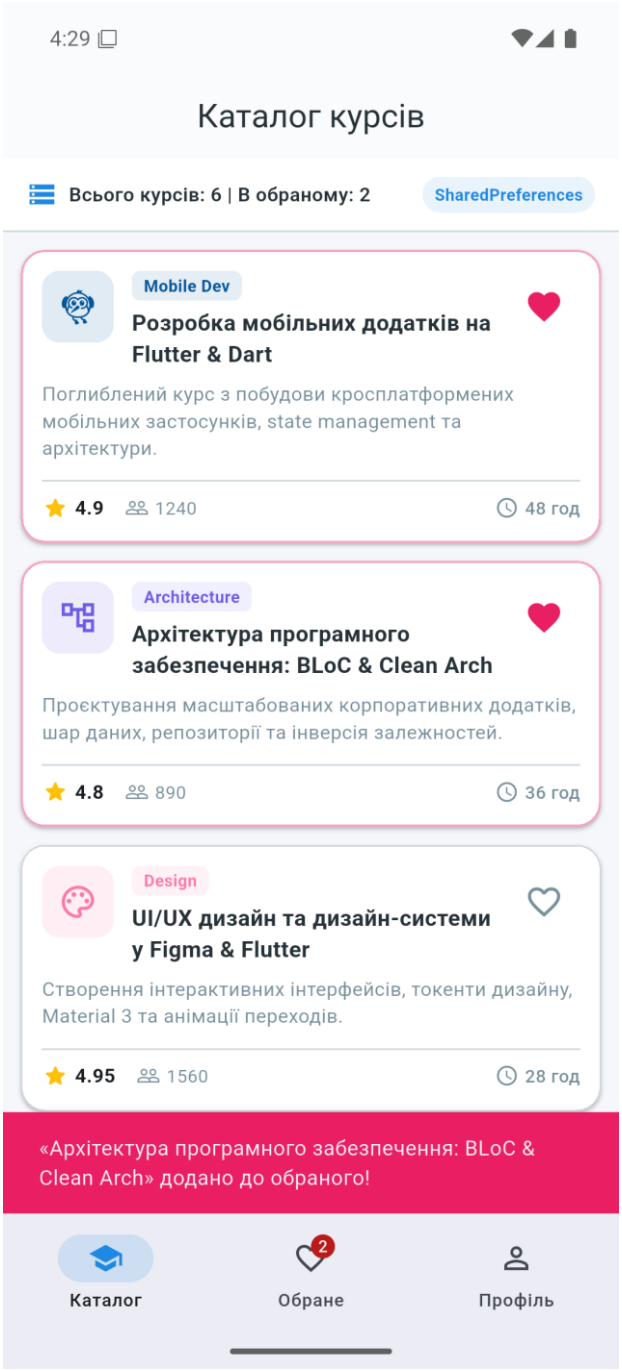


Рис. 3. Каталог курсів із відміченими улюбленими курсами (додавання ID в SharedPreferences)

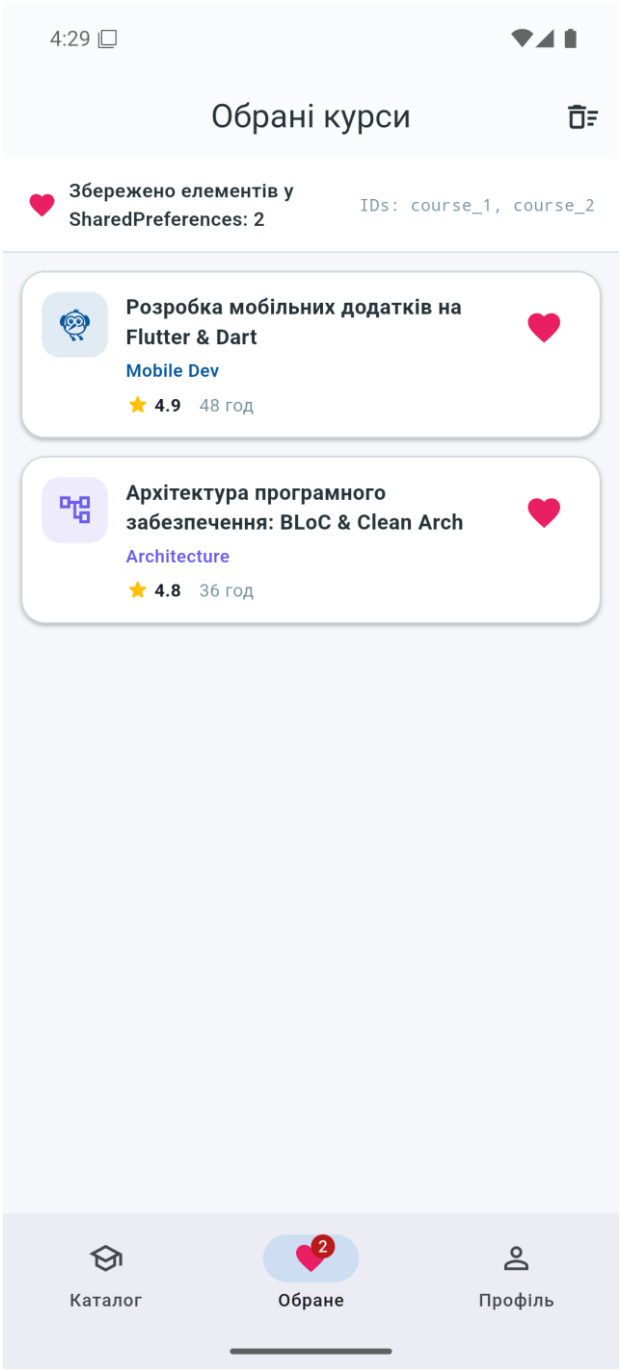


Рис. 4. Екран FavoritesScreen: фільтрація елементів виключно за збереженими favorites ids

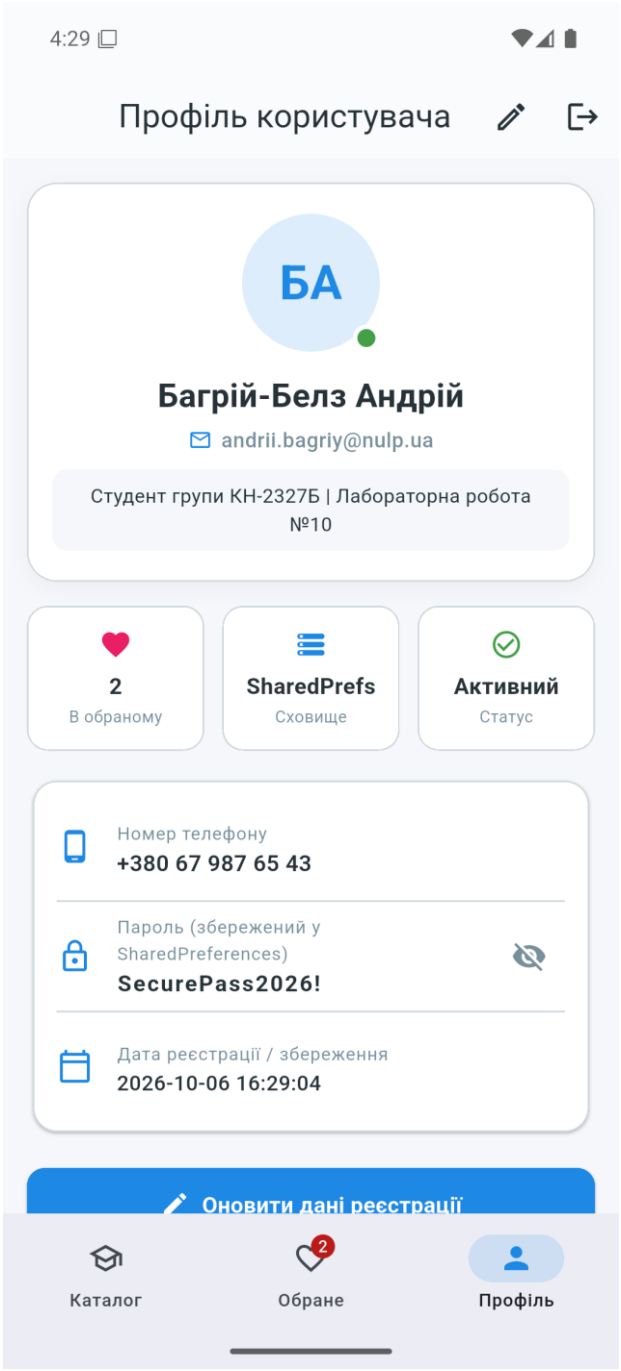


Рис. 5. Екран ProfileScreen: відображення збережених даних користувача, пароля та статистики

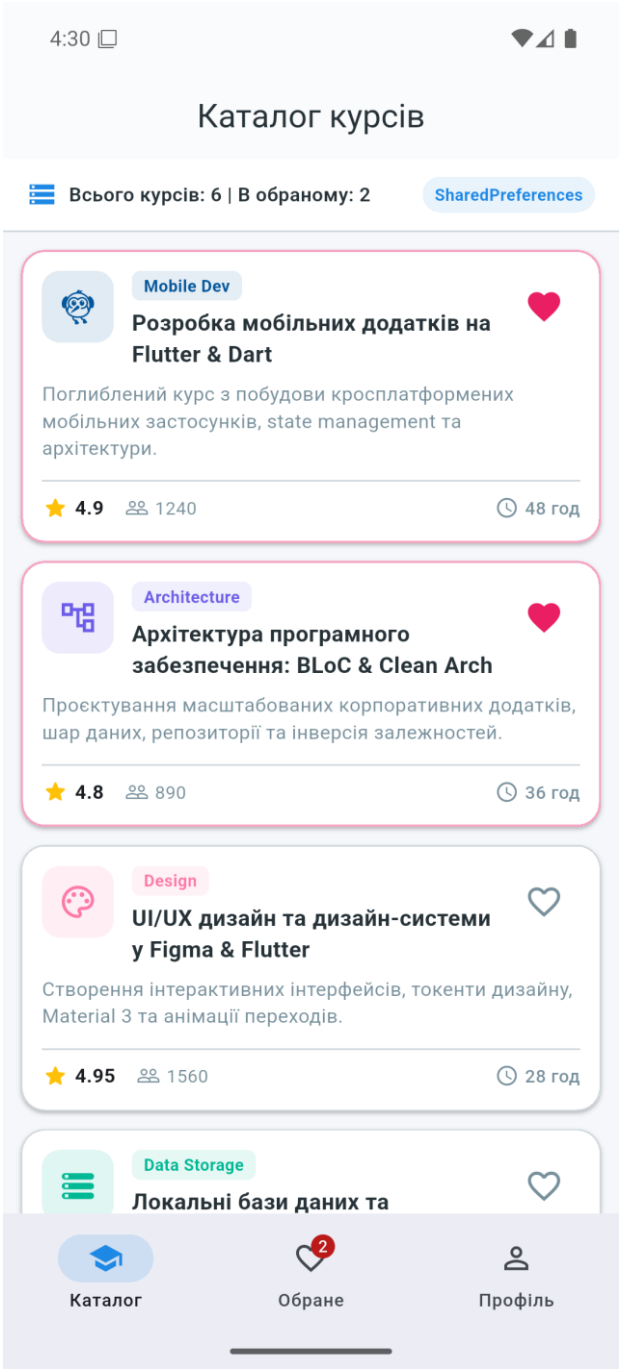


Рис. 6. Перевірка після перезапуску (п. 10 ТЗ): збереження профілю та списку обраного відновлено автоматично

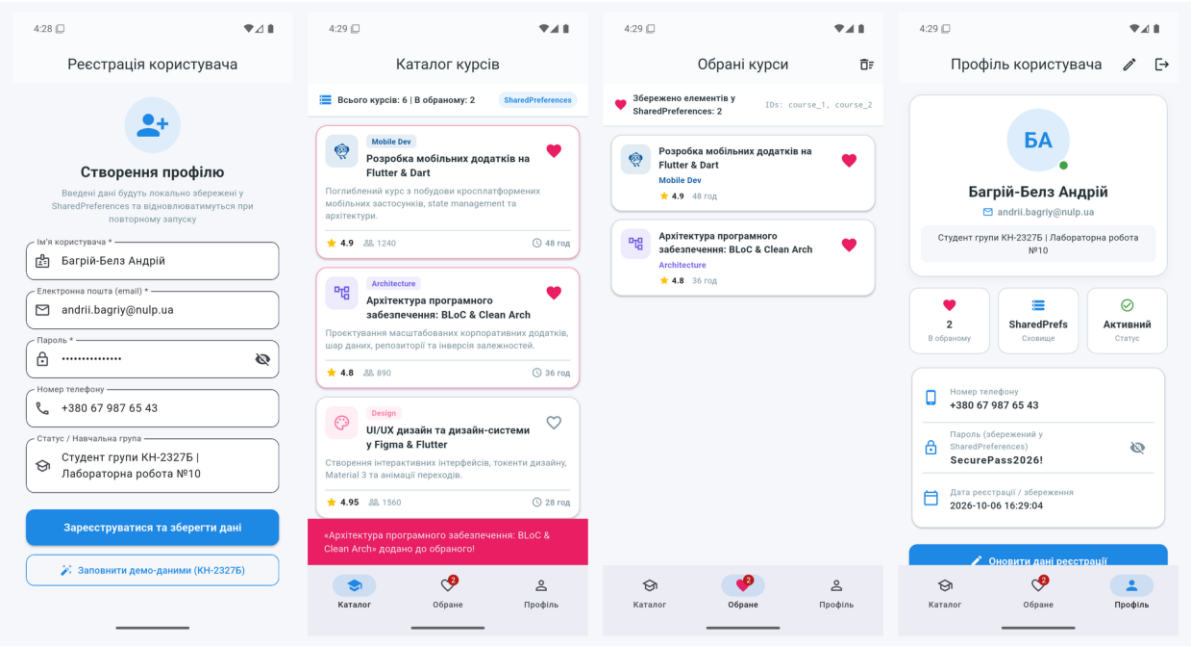


Рис. 7. Загальний огляд ключових екранів взаємодії з локальним сховищем SharedPreferences

Контрольні запитання

1. Що таке локальне збереження даних у Flutter?

Локальне збереження даних (Local Data Persistence) — це механізм довготривалого зберігання структурованої або неструктурованої інформації безпосередньо у внутрішній пам'яті клієнтського мобільного пристрою (пісочниці застосунку) без обов'язкового звернення до віддаленого сервера чи мережі Інтернет. Завдяки цьому забезпечується збереження стану користувача між сесіями (авторизація, налаштування теми, кешовані списки, офлайн-доступ) та миттєве завантаження інтерфейсу під час повторного запуску програми.

2. Які дані в цій лабораторній роботі потрібно зберігати локально?

У межах лабораторної роботи №10 локально зберігаються дві категорії даних:
1) Дані облікового запису користувача, отримані з форми екрана реєстрації: ім’я користувача (key: user_name), адреса електронної пошти (key: user_email), пароль або його хеш (key: user_password), а також супутні параметри форми (номер телефону user_phone, статус user_bio, позначка успішної реєстрації is_registered та дата створення);
2) Дані обраних елементів каталогу: виключно список рядкових ідентифікаторів улюблених курсів (favorites ids, key: favorite_ids), що зберігаються як List<String>.

3. Для чого використовується SharedPreferences?

Пакет SharedPreferences призначений для швидкого та надійного збереження простих даних у форматі пар «ключ-значення» (Key-Value Storage). Під капотом він використовує нативні сховища операційних систем: SharedPreferences у ОС Android (XML-файли у захищеній директорії застосунку) та NSUserDefaults у iOS/macOS. Пакет підтримує базові типи даних: bool, int, double, String та List<String>. Його застосовують для конфігураційних налаштувань, прапорців першого запуску, збереження легких сесійних даних та списків обраних ідентифікаторів.

4. Як зберегти ім’я та email користувача локально?

Збереження здійснюється асинхронним викликом методів setString() об'єкта SharedPreferences:

```
final prefs = await SharedPreferences.getInstance();
await prefs.setString('user_name', name);
```

```
await prefs.setString('user_email', email);
await prefs.setString('user_password', password);
```

Метод повертає Future<bool>, що підтверджує успішність операції дискового запису в асинхронному циклі подій.

5. Як зберегти список favorites ids?

Список favorites ids зберігається за допомогою методу setStringList(), який серіалізує список рядків List<String>:

```
final prefs = await SharedPreferences.getInstance();
await prefs.setStringList('favorite_ids', favoriteIds);
```

Якщо вихідні ідентифікатори мають числовий тип (int), їх попередньо трансформують за допомогою ids.map((e) => e.toString()).toList(), після чого записують у сховище.

6. Чому для обраних елементів доцільно зберігати саме id, а не весь об’єкт?

Збереження лише ідентифікаторів (ID) має принципові архітектурні переваги:

• Мінімізація дискового простору та пам'яті: масив коротких рядків займає лічені байти порівняно з повними JSON-об'єктами;

• Усунення дублювання та конфліктів даних: актуальні властивості сутності (актуальна ціна, опис, рейтинг, статус доступності) надходять з єдиного авторитетного джерела (каталогу, API), унеможливлюючи застарівання даних у закладках;

• Простота синхронізації та валідації: легка перевірка favoriteIds.contains(id) з константною часовою складністю O(1) у Set або швидким проходом у списку;

• Стійкість до змін схеми: модифікація полів моделі CourseItem не порушує вже збережені локально ID користувача.

7. Як отримати збережені дані після запуску застосунку?

Отримання збережених даних виконується під час ініціалізації застосунку за допомогою методів getString(), getStringList() або getBool():

```
final prefs = await SharedPreferences.getInstance();
final name = prefs.getString('user_name') ?? '';
final email = prefs.getString('user_email') ?? '';
final favoriteIds = prefs.getStringList('favorite_ids') ?? [];
```

Обов'язково використовується оператор нульового злиття (??), який захищає від повернення null при першому запуску програми.

8. Для чого використовується initState() при роботі з локальними даними?

Метод initState() життєвого циклу StatefulWidget викликається рівно один раз при первинному створенні та монтуванні екземпляра віджета в дерево елементів. Він є стандартним місцем для ініціалізації сервісів, підписки на події та запуску асинхронного завантаження даних (наприклад, виклику методів loadUserData() та loadFavoriteIds()). Оскільки сам initState() не може бути оголошений як async, всередині нього викликають асинхронну допоміжну функцію або застосовують FutureBuilder, яка після отримання значень викликає setState() для безпечного оновлення інтерфейсу.

9. Як відобразити збережені дані користувача на ProfileScreen?

Для відображення збережених даних на екрані ProfileScreen:

1) Завантажені з SharedPreferences значення передаються у віджет ProfileScreen через параметри конструктора (наприклад, у формі об'єкта UserModel user) або через стан;

2) У методі build() поля моделі підставляються у відповідні віджети відображення:

• Text(user.name, style: AppStyles.title) — ім'я користувача;

• Text(user.email, style: AppStyles.subtitle) — електронна пошта;

• Text(showPassword ? user.password : '••••••••') — пароль з можливістю відкриття;

• Text('${favoritesCount}') — лічильник кількості обраних курсів;

3) При натисканні кнопок редагування або виходу викликаються відповідні колбеки оновлення чи очищення сховища.

10. Як відновити список Favorites після перезапуску застосунку?

Процес відновлення списку Favorites складається з трьох послідовних кроків:

1) Під час старту застосунку викликається await storageService.loadFavoriteIds(), який повертає збережений List<String>;

2) Отриманий список передається у стан головного навігаційного віджета MainNavigationScreen;

3) На екрані FavoritesScreen виконується декларативна фільтрація повного переліку доступних курсів за допомогою методу where:

```
final favoriteCourses = CourseItem.sampleCourses.where((c) =>
favoriteIds.contains(c.id)).toList();
```

Таким чином користувач бачить рівно ті об'єкти, id яких збережені у SharedPreferences, що повністю забезпечує консистентність даних після перезапуску.

11. Які переваги локального збереження даних для мобільного застосунку?

Ключові переваги локального збереження:

• Офлайн-автономність: застосунок залишається функціональним без доступу до Інтернету (перегляд улюблених курсів, профілю);

• Висока швидкість завантаження: читання з вбудованого флеш-накопичувача пристрою триває лічені мілісекунди без мережевих затримок (latency);

• Покращений користувацький досвід (UX): відсутність потреби повторної реєстрації та повторного введення паролів при кожному сеансі;

• Економія трафіку та серверних ресурсів: кешування налаштувань та ідентифікаторів усуває надлишкові мережеві запити.

12. Які помилки можуть виникнути, якщо не перевіряти наявність збережених даних?

Ігнорування перевірки наявності локальних даних може спричинити критичні помилки:

• Помилки Null Safety / NullPointer: виклик методів над null-значеннями (наприклад, prefs.getStringList('favorite_ids')!.length при першому запуску викине Unhandled Exception);

• Рендеринг некоректного UI: показ порожніх або зламаних віджетів, відсутність обов'язкових полів профілю;

• Неправильна маршрутизація: спроба відкрити головний екран для незареєстрованого користувача без початкових даних замість показу екрана реєстрації;

• Розсинхронізація фільтрів: спроба фільтрації колекцій за null-списком або виклик операцій пошуку на порожніх ідентифікаторах.

Висновок

Під час виконання лабораторної роботи №10 було детально досліджено та практично реалізовано механізми локального збереження даних у мобільному Flutter-застосунку з використанням офіційного пакета **shared_preferences**. Розроблено архітектурний сервіс **StorageService**, який забезпечує асинхронне збереження, зчитування, перемикання та очищення параметрів користувача (ім'я, email, пароль, контактні дані) та списку ідентифікаторів улюблених елементів (**favorite_ids**).

Створено повноцінний інтерфейс користувача, що включає екран реєстрації **RegistrationScreen** з валідацією полів та автоматичним переходом, каталог освітніх курсів **CatalogScreen** з інтерактивними кнопками додавання в обране, екран **FavoritesScreen**, який динамічно відображає виключно елементи зі збереженими ID, та профіль користувача **ProfileScreen** з можливістю перегляду та приховування пароля, відображенням лічильників обраного та кнопками редагування й виходу.

Успішність збереження стану перевірено шляхом повного перезапуску застосунку на емуляторі Android 16 (API 36) та комплексом із 7 автоматизованих модульних та віджет-тестів. Підтверджено, що після перезапуску програми дані профілю не втрачаються, екран реєстрації повторно не викликається, а список улюблених курсів автоматично відновлюється за збереженими ідентифікаторами.